



INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING,
KANCHEEPURAM



Web Automation

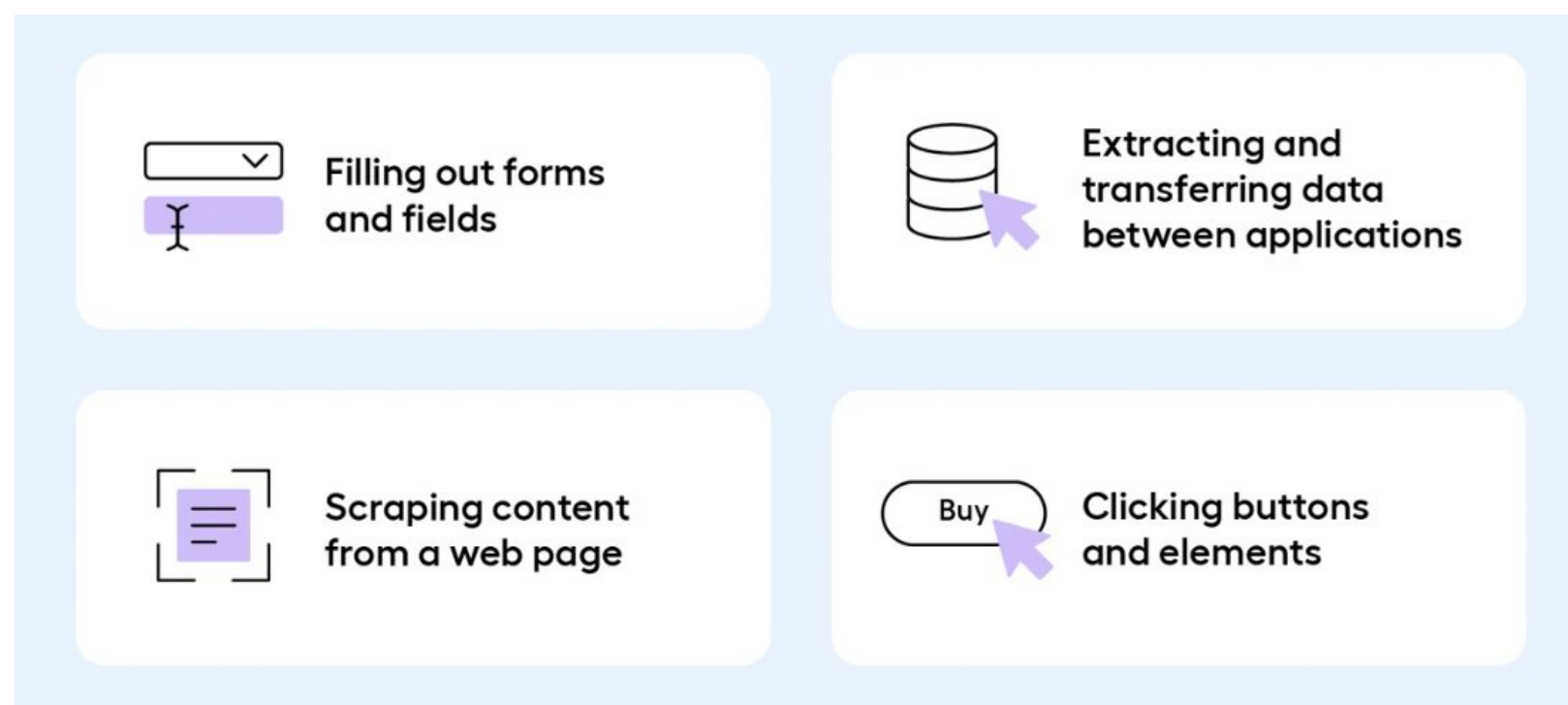
With Selenium (Python)
DEVELOPERS
CLUB





What is Web Automation?

Web automation is the ability to programmatically control a website through its web interface using scripts and tools. This allows organizations to save time and reduce costs by automating processes normally done manually.





Why Use Web Automation?

- **Efficiency:** Automates repetitive tasks (e.g., form filling, testing).
- **Accuracy:** Reduces human error.
- **Speed:** Faster than manual testing/data entry.
- **Scalability:** Handles large-scale data extraction or batch operations.
- **Development Testing:** Automation is integrated during website development to catch issues early. Automated tests provide faster feedback to developers, enabling quick bug detection and reducing the risk of introducing new issues during updates.



Python Libraries Used

- **selenium**: Core library for browser automation.
- **csv**: Reads data from a CSV file to automate form filling.(Built-in)
- **os**: Handles file path operations.(Built-in)
- **time**: Adds delays for stability between actions.(Built-in)



Virtual Environment Setup

(Python) Why is a Virtual Environment?

- An isolated environment for Python projects.
- Prevents dependency conflicts between projects.

```
>> # Download virtual env
>> pip install virtualenv
>>
>> # Create virtual environment
>> python -m venv .venv
>>
>> # Activate it (Windows PowerShell)
>> .venv\Scripts\Activate.ps1
>>
>> # Install packages
>> pip install selenium
>>
>> # Deactivate when done
>> deactivate
```



What is a WebDriver and Why Use It?

Selenium is a browser automation tool, but it cannot directly control browsers on its own. Here's why WebDriver is essential:

Acts as a Bridge Between Selenium & Browsers

- Browsers (Chrome, Firefox, Edge) are complex applications with their own engines.
- WebDriver translates Selenium commands (Python/Java/JS code) into browser-specific actions (e.g., clicking a button, filling a form).

Browser-Specific Communication

- Each browser (Chrome, Firefox) has its own WebDriver implementation:
- Chrome → ChromeDriver
- Firefox → GeckoDriver
- Edge → EdgeDriver
- These drivers understand browser internals and execute commands accordingly.



How to Download ChromeDriver

- Visit **<https://chromedriver.chromium.org/>**
- Download the version matching your installed Chrome.
- Place it in a known location (e.g., C:/Windows/Chromedriver/).
- Set path in code:

```
# Set path to your ChromeDriver
path = "C:/Windows/Chromedriver/chromedriver.exe"

# Set up the Chrome driver
service = Service(path)
driver = webdriver.Chrome(service=service)
```



Common Selenium Functions

Browser Control

Finding elements

Actions on Elements

```
# Opens the specified URL in the browser
driver.get("URL")
# Closes the entire browser session
driver.quit()
# Closes the current browser window
driver.close()
# Reloads the current page
driver.refresh()
```

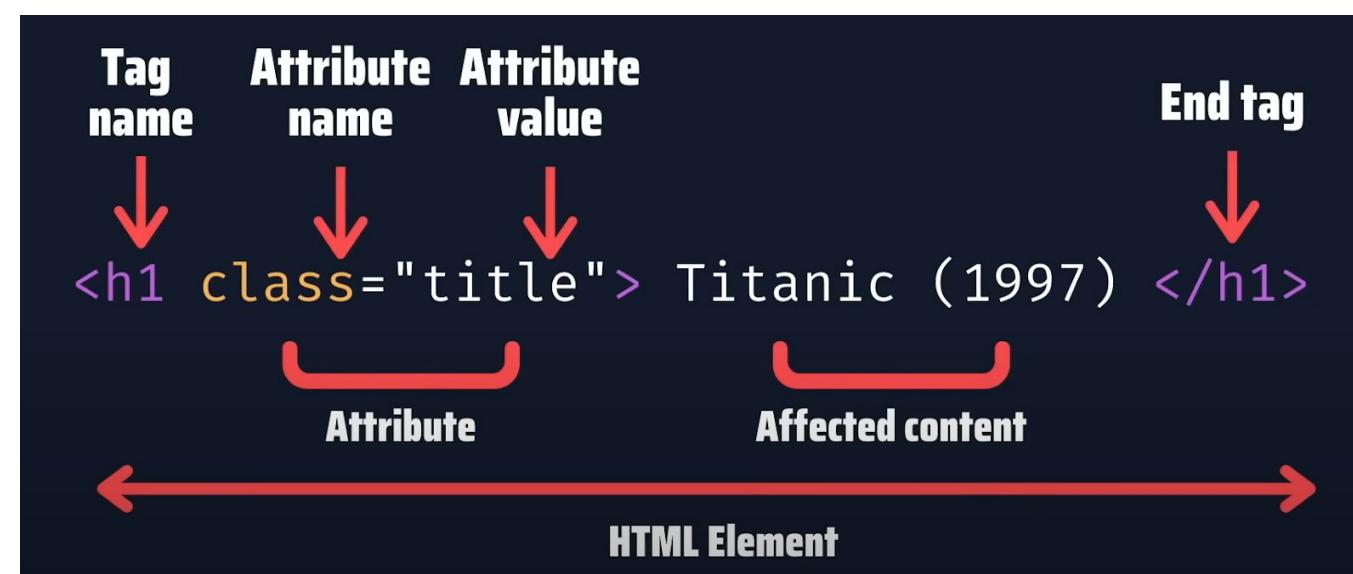
```
# Finds an element by ID
driver.find_element(By.ID, "id")
# By XPath
driver.find_element(By.XPATH, "//tag[@attr='value']")
# By CSS selector
driver.find_element(By.CSS_SELECTOR, "selector")
# By class
driver.find_element(By.CLASS_NAME, "class")
```

```
# Clicks the element
element.click()
# Types text into an input field
element.send_keys("text")
# Clears any text inside the input
element.clear()
# Retrieves the visible text from the element
text = element.text
# Gets a specified attribute's value
attr = element.get_attribute("href")
```



XPath

XPath (XML Path Language) is a **query language** used to **navigate through elements and attributes in XML or HTML documents** — especially useful in **web scraping** and **Selenium** automation. <https://scrapinghub.github.io/xpath-playground/>





Basic XPath Syntax

- **/** → selects the children node
- **//** → selects the matching node at any level
- **//tag** → gives all instances of that tag
- **//tag/text()** → text between the tag
- **//tag[@attribute]** → gives all the instances that have the attribute
- **//tag[@attribute='value']** → gives all the instances that have the attribute and that value
- **//tag[(@attribute='value') or (@attribute)]** → logical OR/AND selection
- **//tag[contains(@attribute, 'value')]** → matches partial attribute values
- **//tag1/tag2** → gives all the instances of tag2 inside tag1



Google Forms Automation

- **Use Case:** Automate the process of filling out and submitting Google Forms using Selenium.
- **Data Source:** CSV file with multiple entries (e.g., Name, Email, Dietary Preference).
- **Steps Involved:**
 - Launch the Google Form in the browser using `driver.get()`.
 - Locate form fields using XPath.
 - Fill text inputs, select radio buttons, choose dropdown values.
 - Submit the form.
 - Click 'Submit another response' to loop for next entry.



Any Questions?



Resources and Roadmap

Setup:

Selenium docs - <https://selenium-python.readthedocs.io/>

Chromedriver download - <https://developer.chrome.com/docs/chromedriver/downloads>

Virtul env docs - <https://virtualenv.pypa.io/en/latest/index.html>

Xpaths:

Xpath cheatsheet - <https://devhints.io/xpath>

Xpath playground - <https://scrapinghub.github.io/xpath-playground/>

Roadmap:

Web automation - <https://www.leapwork.com/blog/web-automation>

Connect with us

Developers Club



Thank You

